



# Revision Notes: Intermediate DSA Lab

## Session on Time Complexity, Space Complexity, Output Prediction, and Debugging

In this session, learners focused on key concepts crucial for writing efficient code and identifying errors quickly. The primary topics covered included Time Complexity (TC), Space Complexity (SC), Output Prediction, and Debugging Techniques.

### Agenda Overview

The session covered:

1. **Iteration Problems** focusing on Time Complexity (TC) and Space Complexity (SC).
2. **Output Prediction Problems** to understand how code execution results can be predicted.
3. **Basic Debugging Problems** to improve error detection and resolution skills 【4:0+source】 .

### Detailed Concepts

#### Time Complexity and Space Complexity

**Time Complexity** is a computational complexity that describes the amount of time it takes to run an algorithm as a function of the length of the input. Different time complexities were analyzed using examples:

- **Constant Time Complexity ( $O(1)$ ):** A function that executes a specific statement, like a print statement, is an example of a constant complexity because it executes once regardless of input size 【4:0+source】 .



【4:0+source】 .

- **Quadratic Time Complexity ( $O(N^2)$ ):** Nested loops are typical examples where the running time increases quadratically as the input size increases 【4:0+source】 .
- **Logarithmic Time Complexity ( $O(\log N)$ ):** This occurs in algorithms that successively divide the problem in half, such as binary search 【4:0+source】 【4:5+source】 .
- **Exponential Time Complexity ( $O(2^N)$ ):** Observed in algorithms where the growth doubles with each addition to the input data set.

**Space Complexity** measures the total amount of memory space that an algorithm or operation takes in terms of the size of the input to the program. Typically logarithmic and linear complexities consume constant space 【4:0+source】 .

## Output Prediction

Involves understanding the result of code execution. This requires a proper understanding of the logic structure and sequence of code execution. Predicting outputs is essential for validating and testing code reliability 【4:15+source】 .

## Debugging Techniques

Debugging is a process of fixing errors and enhancing code reliability. Common debugging tasks include:

- Correcting logical errors such as infinite loops due to improper loop conditions or updates 【4:2+source】 .
- Fixing bounded errors such as array index out of bounds{issues 【4:19+source】 .
- Addressing errors due to undeclared variables or functions 【4:18+source】 .

## Analogies and Clarifications



having different growth rates. For instance, understanding  $n^2$  as quadratic like quadratic equations helps comprehend its broader implications 【4:0+source】 【4:3+source】 .

- *Logarithm Explanation with Powers:* Understanding how logarithmic iterations relate to powers helps in visualizing how operations like binary search work 【4:10+source】 【4:13+source】 .

## Applied Learning

The session emphasizes applied learning with practical coding problems that demanded finding iterations, predicting outputs, and debugging existing code. The learners were encouraged to engage actively by attempting quizzes and solving provided coding challenges 【4:7+source】 【4:8+source】 【4:15+source】 .

Remember, understanding these fundamentals in depth improves your ability to write more efficient, optimized, and error-free code which is crucial for tackling real-world challenges in software engineering.